TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:
Enero - Junio 2026

CARRERA:
Ingeniería en Sistemas Computacionales

MATERIA:
Patrones de diseño

TÍTULO ACTIVIDAD:
Actividad de evidencia 2_U2 Singleton

UNIDAD A EVALUAR:
UNIDAD II

NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO:
RODRIGUEZ GOMEZ BRYAN ALEJANDRO - 22210345

NOMBRE DEL MAESTRO (A):
MARIBEL GUERRERO LUIS

09/03/2026

1. Diagrama UML

```
+----------------------+
|       Operador       |
+----------------------+
| - Id_Operador : int  |
| - Nombre : string    |
+----------------------+
| + Operador(id, nombre) |
| + AtiendeEmergencia(tipoEmergencia) : void |
+----------------------+
           usa
            |
            v
+-----------------------------+
|        Central_911          |
+-----------------------------+
| - _instance : Central_911   |
| - _lock : object            |
| - Central : string          |
+-----------------------------+
| - Central_911()             |
| + Obtener_Instancia() : Central_911 |
| + ConectarLlamada(Operador, string) |
+-----------------------------+


            usa
             |
             v

+----------------------+
|       Program        |
+----------------------+
| + Main(args) : void  |
+----------------------+
```

2. Código Ejecutable

```
Singleton                                   Singleton.Program
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Singleton
{
    0 references
    internal class Program
    {
        0 references
        static void Main(string[] args)
        {
            Central_911 Llamada1 = Central_911.Obtener_Instancia();
            Central_911 Llamada2 = Central_911.Obtener_Instancia();
            Central_911 Llamada3 = Central_911.Obtener_Instancia();
            Central_911 Llamada4 = Central_911.Obtener_Instancia();

            Operador op1 = new Operador(1, "Laura");
            Operador op2 = new Operador(2, "Carlos");
            Operador op3 = new Operador(3, "Arturia");
            Operador op4 = new Operador(4, "Daniel");

            Llamada1.ConectarLlamada(op1, "Incendio");
            Llamada2.ConectarLlamada(op2, "Violeta");
            Llamada3.ConectarLlamada(op4, "Accidente");
            Llamada4.ConectarLlamada(op3, "Intento de suicidio");

            Console.WriteLine(ReferenceEquals(Llamada1, Llamada2));
            Console.WriteLine(ReferenceEquals(Llamada1, Llamada3));
            Console.WriteLine(ReferenceEquals(Llamada1, Llamada4));

            Console.ReadKey();
        }
    }
}
```

I. .-El atributo estático de la instancia

se encuentra dentro de la clase “Central_911” y se trata de:

```
private static Central_911 _instance;
```

Esto significa que significa que pertenece a la clase y no a un objeto y guarda la única instancia de la clase Central_911

II. .-El constructor

El constructor se encuentra dentro de la misma clase:

```
1 reference
private Central_911()
{
    Central = "Central 911";
}
```

El constructor está declarado como private lo que significa que no puede ser llamado desde fuera de la clase y solo la propia clase “Central_911” puede crear su instancia.

III. ¿Por qué no se pueden crear múltiples objetos?

No se pueden crear por lo mencionado anteriormente y es que el constructor está declarado como privado

3. Conclusión

El uso del patrón de diseño Singleton permite que todas las llamadas de emergencia se gestionan a partir de una sola central. De esta manera se evita la creación de múltiples objetos de la misma clase, lo cual asegura consistencia en el sistema y un mejor control de los recursos.